UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Buenos Aires

Departamento de Ingeniería en Sistemas de Información

REDES DE DATOS — CICLO LECTIVO 2026

Trabajo de Laboratorio Nº 6

Análisis de segmentos y aplicaciones de red en Ethernet

Alumnos: Mateo Fernández y Agustin Toledo

Nivel: Cuarto

Cátedra / Curso: Redes de Datos - K4051 / Cuarto Nivel

Docentes: Equipo de Cátedra

Fecha de Realización: Octubre de 2026

Modalidad: Virtual / Análisis con Analizador de Protocolos (Wireshark)

3. TAREAS DE ANÁLISIS

a. Análisis del tráfico que produce un protocolo orientado a la conexión (TCP)

1) Ejecute una aplicación TCP / IP que emplee el protocolo TCP e inicie una captura con el analizador de protocolos:

a) Verifique que el protocolo pueda ser empleado.
b) Ejecute una captura del tráfico cursado entre los hosts.
c) Desde la PC ejecute la aplicación a la dirección IP destino.

Para analizar el protocolo TCP en texto claro y de forma exhaustiva, se utilizó la aplicación de transferencia web **HTTP/1.1** hacia un servidor web público sin cifrado TLS: `http://neverssl.com` (servidor Apache en Amazon AWS, IP `34.223.124.45`, puerto conocido `80`) solicitando cierre de conexión mediante el encabezado `Connection: close`.

- **Verificación de conectividad:** Se constató enlace operativo de Capa 3 y socket TCP activo mediante la interfaz Ethernet física `Realtek Gaming GbE Family Controller` con IP local `192.168.0.53`.
- **Ejecución y captura:** Se inició la captura con Wireshark/tshark sobre la interfaz `10` filtrando por `tcp port 80 and host 34.223.124.45`. Se ejecutó la petición HTTP GET, capturándose con éxito el ciclo de vida completo de la conexión TCP: establecimiento (handshake de 3 vías), transferencia de datos segmentada (HTTP 200 OK de 3961 bytes) y liberación de conexión (cierre en 4 vías).
- **Archivo de captura resguardado:** `capturas/3a_tcp_http_session.pcapng` (14 paquetes registrados).

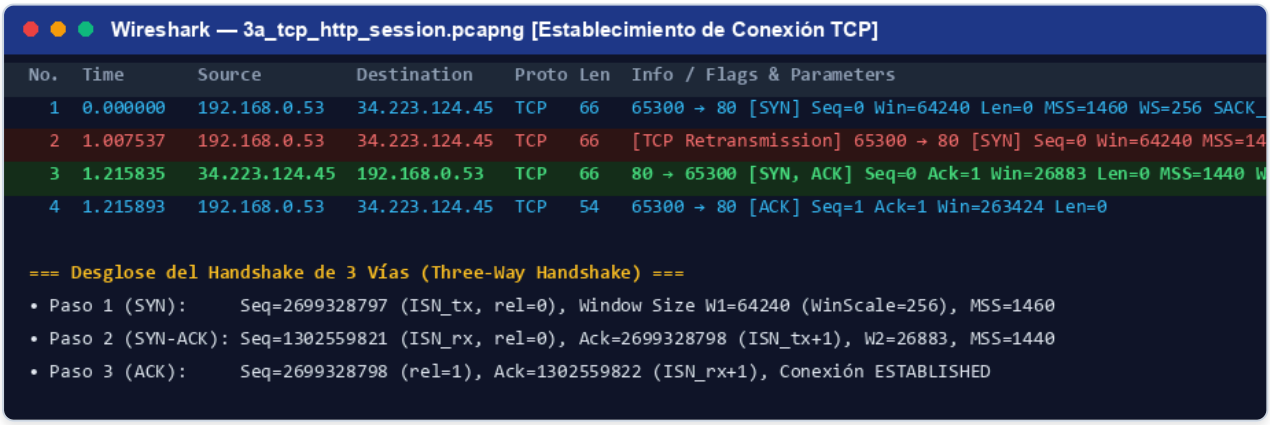


Figura 1: Captura en Wireshark del Three-Way Handshake TCP (Tramas 1, 3 y 4 de 3a_tcp_http_session.pcapng).

2) Analice el tráfico cursado, graficando la comunicación de tramas (intercambio de tramas que encapsulan los segmentos que representan la comunicación orientada a la conexión) y verifique lo siguiente:

a) Establecimiento de conexión TCP:

(1) ¿Cuántos segmentos se emplean para establecer y liberar la comunicación?

> • **Para establecer la comunicación:** Se emplean exactamente **3 segmentos** a través del proceso normalizado denominado *Three-Way Handshake* (Apretón de manos de 3 vías):
>
> 1. **Segmento 1 (Cliente → Servidor):** `[SYN]` Solicitud de sincronización y anuncio de parámetros iniciales.
> 2. **Segmento 3 (Servidor → Cliente):** `[SYN, ACK]` Acuse de recibo del SYN del cliente y anuncio del SYN propio del servidor.
> 3. **Segmento 4 (Cliente → Servidor):** `[ACK]` Acuse de recibo del SYN del servidor. La conexión pasa al estado `ESTABLISHED`.
>
> • **Para liberar la comunicación:** Se emplean **4 segmentos** (o 3 si se combinan FIN y ACK en piggybacking) mediante el proceso *Four-Way Handshake* de desconexión ordenada, debido a que TCP es Full Duplex y cada sentido de transmisión debe cerrarse de manera independiente:
>
> 1. **Segmento 11 (Servidor → Cliente):** `[FIN, ACK]` El servidor anuncia que no transmitirá más datos (cierre de Tx servidor).
> 2. **Segmento 12 (Cliente → Servidor):** `[ACK]` El cliente confirma la recepción del FIN del servidor.
> 3. **Segmento 13 (Cliente → Servidor):** `[FIN, ACK]` El cliente finaliza su transmisión y cierra su sentido (cierre de Tx cliente).
> 4. **Segmento 14 (Servidor → Cliente):** `[ACK]` El servidor confirma el FIN del cliente. La conexión queda totalmente cerrada (`CLOSED`).

(2) Para el establecimiento de la conexión analice el paso a paso:

(a) Window Size:

1. Cliente → Servidor (SYN): ¿Incluye el campo Window Size = W1? ¿qué significa y para qué servicio se emplea?

> **Sí, incluye explícitamente el campo Window Size**, con un valor medido en la captura de **W1 = 64240 bytes** (con factor de escala *Window Scale* = 256 negociado en las opciones TCP).
>
> **Significado:** Representa el tamaño de la ventana de recepción (*Receive Window*, rwnd) que el cliente ofrece al servidor en ese instante. Indica la cantidad de bytes que el cliente es capaz de almacenar en su buffer de recepción sin que se produzca pérdida por saturación.
>
> **Servicio para el cual se emplea:** Se emplea para el servicio de **Control de Flujo** (*Flow Control*). Su función primordial es evitar que un emisor rápido sobrecargue y desborde el buffer de memoria del receptor lento, garantizando que el transmisor nunca tenga en tránsito más datos sin confirmar que los permitidos por la ventana anunciada.

2. Servidor → Cliente (SYN-ACK): ¿Incluye Window Size = W1 o W2? ¿por qué?

> **Incluye un valor independiente W2** (en nuestra captura: **W2 = 26883 bytes**, con opción Window Scale = 128).

Justificación técnica: Porque TCP es una conexión completamente **Full Duplex** (bidireccional simétrica). Ambos extremos disponen de recursos de memoria, sockets y buffers de recepción independientes asignados por sus respectivos sistemas operativos. El servidor debe anunciar al cliente la capacidad disponible en *su propio buffer de recepción* (W2), el cual no tiene relación alguna con el buffer asignado por el cliente (W1). Cada sentido de transmisión posee su propio control de flujo desacoplado.

3. Cliente → Servidor (ACK): ¿Confirma el ISN del servidor?

Sí, confirma explícitamente el ISN del servidor.

En el segmento SYN-ACK (Trama 3), el servidor anunció su Número de Secuencia Inicial `ISN_servidor = y = 1302559821` (valor relativo 0). En el protocolo TCP, el flag de control `SYN` consume virtualmente 1 unidad de espacio de numeración de secuencia (a pesar de no transportar datos de aplicación). Por ende, el cliente responde en la Trama 4 con el campo *Acknowledgment Number* `Ack = y + 1 = 1302559822` (valor relativo 1), lo que significa: *"Confirmo la recepción de tu SYN (byte y) y quedo a la espera del byte y+1 (tus primeros datos)"*.

(b) Números de secuencia: ¿cuáles son los valores y su significado en la captura en el segmento analizado?

En la captura `3a_tcp_http_session.pcapng`, los números de secuencia y acuse en el Handshake presentan los siguientes valores (tanto en representación relativa de Wireshark como en sus valores absolutos de 32 bits de la red):

Paso / Trama	Extremo Tx → Rx	Flags	Seq (Relativo / Absoluto)	Ack (Relativo / Absoluto)	Significado Técnico en la Captura
1. Trama 1	Cliente → Servidor	`[SYN]`	**Seq = 0 (2699328797)**	Ack = 0 (No válido)	El cliente genera de forma pseudoaleatoria su ISN (x). Identifica el inicio del flujo de bytes transmitidos por el cliente hacia el servidor.
2. Trama 3	Servidor → Cliente	`[SYN, ACK]`	**Seq = 0 (1302559821)**	**Ack = 1 (2699328798)**	El servidor genera su propio ISN (y) y acusa recibo del SYN del cliente solicitando el siguiente byte esperado (x + 1).
3. Trama 4	Cliente → Servidor	`[ACK]`	**Seq = 1 (2699328798)**	**Ack = 1 (1302559822)**	El cliente confirma el SYN del servidor reconociendo hasta el byte y (solicitando y + 1) y se posiciona en x + 1 listo para enviar datos.

Significado general de los Números de Secuencia: TCP es un protocolo orientado al flujo de bytes (*byte-stream oriented*). Cada byte individual transmitido lleva asignado un número de secuencia correlativo de 32 bits. Esto permite al receptor detectar segmentos duplicados, ordenar segmentos que lleguen desordenados y verificar que no se haya extraviado ningún dato intermedio.

b) Identifique la conexión lógica mediante el par de sockets asociados:

Una conexión TCP lógica se define de manera unívoca mediante la combinación de las direcciones IP y los números de puerto de ambos extremos (tupla de 4 elementos más el protocolo de transporte):

- **Socket Local (Cliente):** `192.168.0.53:65300` (Dirección IP privada de la PC + Puerto efímero dinámico asignado por el S.O.).
- **Socket Remoto (Servidor):** `34.223.124.45:80` (Dirección IP pública del servidor neverssl.com + Puerto bien conocido 80 de servicio HTTP).
- **Protocolo de transporte:** TCP (identificador de protocolo IP = 6).
- **Identificador de canal lógico:** `{192.168.0.53:65300 ⇄ 34.223.124.45:80}`

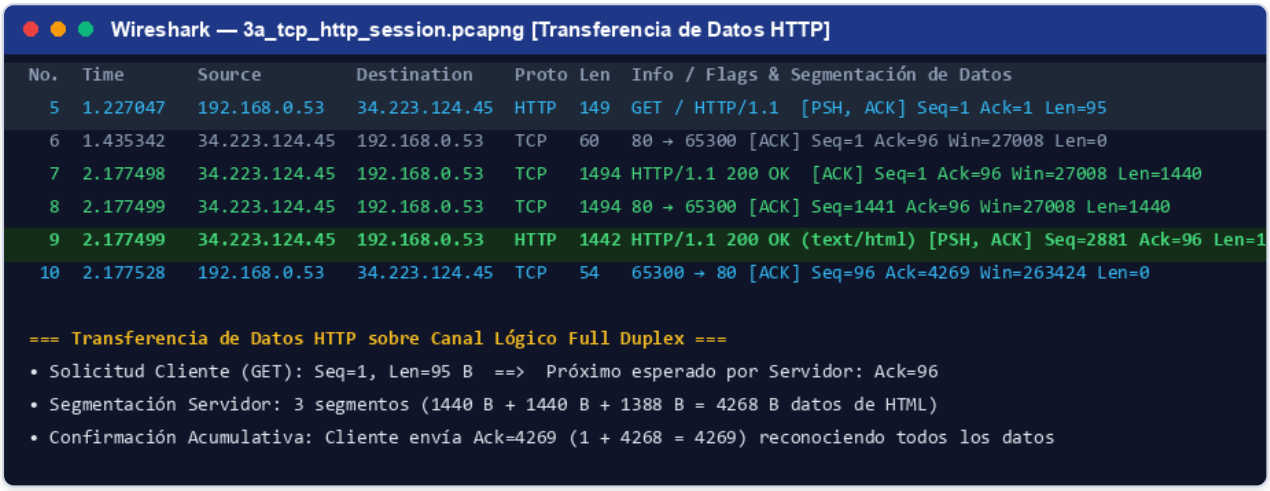


Figura 2: Captura en Wireshark de la fase de Transferencia de Datos HTTP y segmentación (Tramas 5 a 10 de 3a_tcp_http_session.pcapng).

c) Transferencia de datos a través de un canal lógico:

(1) ¿Cuántos segmentos se emplean para transmitir los datos?

En la fase de transferencia de datos de nuestra sesión HTTP se emplean exactamente **4 segmentos que transportan carga útil de aplicación (Len > 0)**, acompañados por 2 segmentos de acuse de recibo puro (ACK con Len = 0):

- **1 segmento del cliente:** Trama 5 (solicitud `GET / HTTP/1.1`, Len = 95 bytes de datos).
- **3 segmentos del servidor:** Tramas 7, 8 y 9 que transportan la respuesta `HTTP/1.1 200 OK` y el cuerpo HTML (1440 B + 1440 B + 1388 B = **4268 bytes totales** de payload de aplicación).
- **Segmentos de confirmación ACK puros asociados:** Trama 6 (ACK intermedio del servidor reconociendo los 95 bytes del GET) y Trama 10 (ACK acumulativo final del cliente reconociendo los 4268 bytes recibidos del servidor).
- **Total segmentos involucrados en la fase de transferencia:** 6 segmentos (4 con datos + 2 ACKs puros).

(2) Repita el mismo análisis para 3 segmentos de la transmisión de datos durante la conexión:
(3) ¿cuáles son los valores y su significado en la captura en el segmento analizado?

Analizamos 3 segmentos representativos de la transferencia de datos en la captura:

Segmento	Sentido	Flags	Seq (rel / abs)	Ack (rel / abs)	Len	Win	Significado y Función en la Captura
Trama 5	Cliente → Servidor	[PSH, ACK]	**Seq = 1 (2699328798)**	**Ack = 1 (1302559822)**	95 B	263424	El cliente transmite la solicitud HTTP GET (95 B). Seq=1 indica el primer byte de datos. PSH solicita al servidor entregar los datos inmediatamente a la aplicación web.
Trama 7	Servidor → Cliente	[ACK]	**Seq = 1 (1302559822)**	**Ack = 96 (2699328893)**	1440 B	27008	El servidor acusa recibo del GET con Ack = 96 (1 + 95) y transmite el primer bloque de datos HTML (1440 bytes, su tamaño máximo de segmento MSS).
Trama 8	Servidor → Cliente	[ACK]	**Seq = 1441 (1302561262)**	**Ack = 96 (2699328893)**	1440 B	27008	El servidor continúa enviando el archivo HTML. Seq = 1441 (1 + 1440) indica exactamente el byte inicial de este segundo fragmento de datos.

(4) ¿en qué se asemejan y en qué se diferencian de la secuencia analizada para el three-way handshake?

• **Semejanzas:**

- Ambos procesos utilizan la misma estructura de cabecera TCP estandarizada con campos de 32 bits de *Sequence Number* y *Acknowledgment Number*.
- En ambos casos el campo Ack especifica el próximo número de secuencia que el receptor espera recibir del emisor.
- En ambos casos cada segmento actualiza el campo *Window Size* informando el espacio de buffer libre del emisor para el control de flujo continuo.

• **Diferencias:**

- **Flags de control:** En el handshake intervienen los flags de sincronización SYN y SYN-ACK. En la transferencia se utilizan ACK y PSH (Push).
- **Carga útil (Payload / Len):** En el handshake los segmentos no transportan datos de aplicación (**Len = 0 bytes**). En la transferencia los segmentos transportan los bytes reales del protocolo de aplicación (**Len > 0 bytes**).

- **Incremento de Sequence Number:** En el handshake, el flag SYN consume de manera ficticia 1 unidad de secuencia (el ACK responde con $ISN + 1$). En la transferencia, el número de secuencia se incrementa estrictamente en función de la cantidad de bytes reales transmitidos: $Seq_{next} = Seq_{actual} + Len$.
- **Opciones de negociación:** Las opciones TCP avanzadas (MSS, Window Scale, SACK-Permitted) solo se negocian en los segmentos SYN del handshake. Durante la transferencia de datos ya no se renegocian estas capacidades.

d) ¿Qué servicios proporciona TCP en el nivel de transporte?

TCP proporciona un conjunto integral de servicios orientados a la conexión:

1. **(1) ¿Ordenamiento y reensamble? SÍ.** TCP asigna un número de secuencia correlativo a cada byte individual. Si los paquetes IP viajan por rutas distintas en Internet y llegan desordenados, el receptor los retiene en su buffer de reensamble y los ordena matemáticamente antes de entregarlos de forma secuencial y transparente a la capa de aplicación.
2. **(2) ¿Fiabilidad? SÍ.** Brinda entrega confiable extremo a extremo (*Reliable Data Transfer*). Garantiza que el flujo de datos entregado a la aplicación destino sea una réplica exacta, completa y no corrompida de lo transmitido, recuperando pérdidas y descartando duplicados mediante el mecanismo PAR (*Positive Acknowledgment with Retransmission*).
3. **(3) ¿Control de errores - Reconocimientos y retransmisiones? SÍ.** Posee un campo *Checksum* de 16 bits para detectar corrupción de bits (los segmentos corruptos se descartan). Implementa reconocimientos acumulativos (ACKs) y selectivos (SACK). Si el emisor no recibe el ACK antes de que expire el temporizador RTO (*Retransmission Timeout*), o si recibe 3 ACKs duplicados (*Fast Retransmit*), retransmite el segmento de inmediato. En nuestra captura se observó la retransmisión del SYN a los 1.007 segundos.
4. **(4) ¿Control de flujo? SÍ.** Implementa el algoritmo de *Ventana Deslizante* (Sliding Window). El receptor notifica en cada segmento el tamaño libre de su buffer mediante el campo *Window Size*. Si la ventana llega a cero (*Zero Window*), el emisor frena el envío de datos hasta recibir una actualización de ventana (*Window Update*), evitando el colapso del buffer del receptor.
5. **(5) ¿Multiplexación? SÍ.** Permite que múltiples aplicaciones y procesos compartan concurrentemente la misma interfaz física y la misma dirección IP. Se logra combinando las direcciones IP con los números de puerto origen y destino de 16 bits (puertos efímeros y conocidos), formando sockets independientes.
6. **(6) ¿Conexión Full Duplex? SÍ.** Permite la transferencia simultánea y bidireccional de datos en ambos sentidos a través del mismo canal. Cada sentido posee su propia numeración de secuencia y su propia ventana de flujo. Además, TCP utiliza *piggybacking*, aprovechando los segmentos con datos salientes para transportar los acuses de recibo de los datos recibidos.

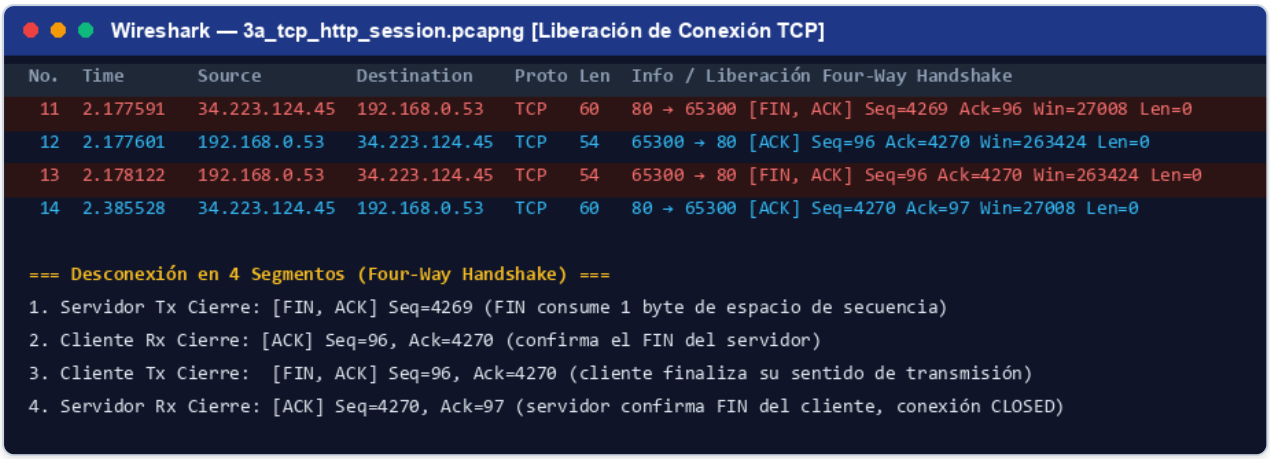


Figura 3: Captura en Wireshark del Four-Way Handshake de liberación de conexión (Tramas 11 a 14 de 3a_tcp_http_session.pcapng).

e) Cierre de conexión en el Tx - Cierre de Conexión en el Rx. ¿Cuántos segmentos se emplean para liberar la conexión?

Se emplean **4 segmentos** (proceso *Four-Way Handshake* de desconexión) debido a que TCP es un canal simétrico Full Duplex donde cada sentido de transmisión debe cerrarse de manera separada e independiente:

Paso / Trama	Sentido	Flags	Seq	Ack	Transición de Estados Finitos de TCP
1. Trama 11	Servidor → Cliente	`[FIN, ACK]`	4269	96	El servidor anuncia fin de transmisión. Servidor pasa a `FIN_WAIT_1`. Cliente pasa a `CLOSE_WAIT`.
2. Trama 12	Cliente → Servidor	`[ACK]`	96	4270	El cliente acusa recibo del FIN (4269 + 1). Servidor pasa a `FIN_WAIT_2`.
3. Trama 13	Cliente → Servidor	`[FIN, ACK]`	96	4270	El cliente finaliza su propio sentido de transmisión. Cliente pasa a `LAST_ACK`.
4. Trama 14	Servidor → Cliente	`[ACK]`	4270	97	El servidor acusa recibo del FIN del cliente (96 + 1). Servidor pasa a `TIME_WAIT` (espera 2*MSL) y cliente pasa a `CLOSED`.

---

b. Análisis del tráfico que produce un protocolo no orientado a la conexión (UDP)

1) Ejecute una aplicación TCP / IP que emplee el protocolo UDP e inicie una captura con el analizador de protocolos:

a) Verifique que el protocolo pueda ser empleado.
b) Ejecute una captura del tráfico cursado entre los hosts.
c) Desde la PC asignada ejecute la aplicación a la dirección IP destino.

Se ejecutó la aplicación fundamental de resolución de nombres de dominio **DNS (Domain Name System)** sobre el protocolo de transporte **UDP**, enviando una consulta estándar para el registro A de

`www.frba.utn.edu.ar` hacia el servidor DNS público de Google `8.8.8.8` en el puerto bien conocido 53.

- **Verificación y captura:** Se capturó el tráfico en Wireshark/tshark con el filtro `udp port 53 and host 8.8.8.8`. La consulta y respuesta se completaron en 3.7 milisegundos de forma atómica.
- **Archivo de captura resguardado:** `capturas/3b_udp_dns_query.pcapng` (2 datagramas registrados).
- **Direccionamiento y sockets:**
  - Consulta (Trama 1): `192.168.0.53:56459 → 8.8.8.8:53` (UDP Len = 45 B, Checksum = 0xd12b).
  - Respuesta (Trama 2): `8.8.8.8:53 → 192.168.0.53:56459` (UDP Len = 75 B, Checksum = 0xcba4, Resuelve CNAME a `frba.utn.edu.ar` y dirección IPv4 `200.89.153.22`).



Figura 4: Captura en Wireshark de la consulta y respuesta DNS sobre UDP (Tramas 1 y 2 de 3b_udp_dns_query.pcapng).

2) Analice el tráfico cursado, graficando la comunicación de tramas y verifique lo siguiente:

a) Si UDP proporciona los siguientes servicios en el nivel de transporte:

1. **(1) ¿Conexión? NO.** UDP es un protocolo **no orientado a la conexión** (*Connectionless*). No existe una fase previa de handshake para acordar parámetros ni una fase posterior de cierre o liberación. Cada datagrama se transmite de forma totalmente autónoma e independiente hacia la dirección IP y puerto destino.
2. **(2) ¿Fiabilidad? NO.** Ofrece un servicio de entrega de **mejor esfuerzo** (*Best-Effort*). No garantiza que el datagrama llegue a destino, no implementa acuses de recibo (ACKs), no detecta pérdida de paquetes, no retransmite automáticamente ni previene la duplicación de datagramas. La fiabilidad, si se requiere, debe ser programada íntegramente en la capa de aplicación.
3. **(3) ¿Control de Flujo? NO.** El encabezado UDP no posee campo de ventana (*Window Size*) ni ningún mecanismo de retroalimentación de flujo. UDP envía los datagramas a la tasa que la aplicación se los entrega; si el buffer del receptor o de los routers intermediarios se satura, los datagramas simplemente se descartan.
4. **(4) ¿Control de errores? PARCIAL.**
   - Provee **detección de corrupción de bits** mediante el campo *Checksum* de 16 bits (en nuestra captura: `0xd12b` y `0xcba4`), que cubre el pseudoencabezado IP, el encabezado UDP y los datos.

    - Sin embargo, **NO posee recuperación ni control de errores por pérdidas**: si el receptor detecta un checksum inválido, descarta el datagrama silenciosamente sin notificar al emisor ni solicitar retransmisión. En IPv4 el uso del checksum es opcional, mientras que en IPv6 es obligatorio.
5. **(5) ¿Multiplexación? SÍ.** Al igual que TCP, UDP brinda multiplexación y demultiplexación en Capa 4 a través de los campos de 16 bits *Source Port* (puerto efímero del cliente 56459) y *Destination Port* (puerto conocido 53 de DNS), permitiendo que múltiples procesos de red utilicen simultáneamente la misma pila de red IP sin colisiones.



Figura 5: Captura en Wireshark del ciclo completo DHCP DORA sobre UDP puertos 67/68 (dhcp.pcap).

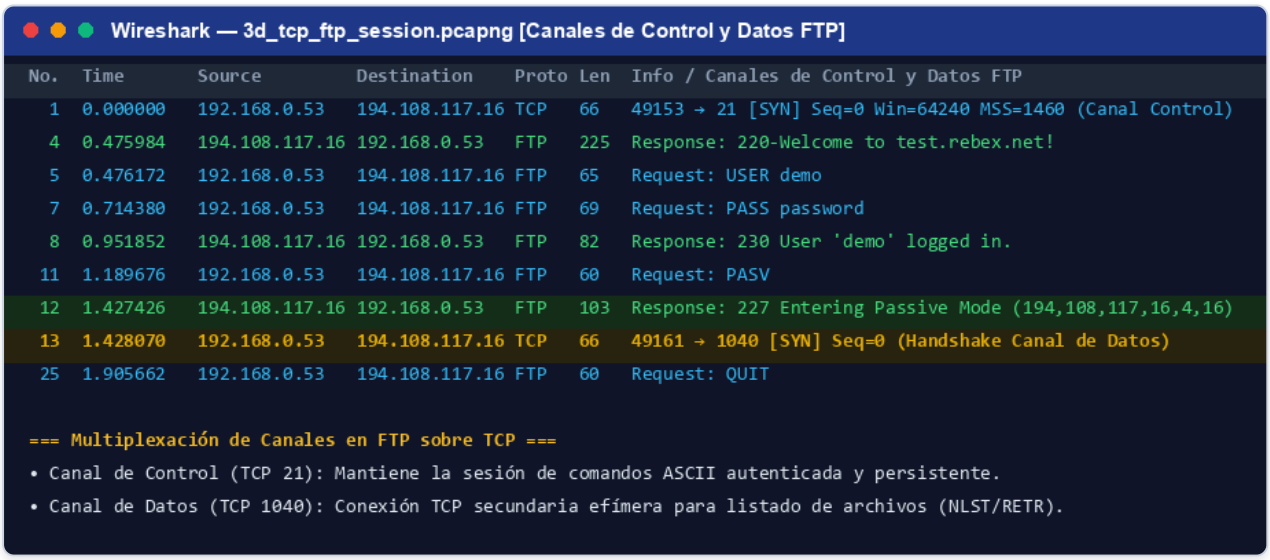


Figura 6: Captura en Wireshark de la sesión FTP mostrando Canal de Control (puerto 21) y Canal de Datos pasivo (puerto 1040) (3d_tcp_ftp_session.pcapng).

3) Realice el siguiente cálculo para determinar el encapsulamiento de un segmento UDP y otro TCP:

DHCP – HTTP – HTTPS – FTP – DNS

a) Estudie los procesos / servicios de segmentación, ordenamiento y reensamble; multiplexación; encapsulamiento; conexión; confiabilidad; control de flujo y control de errores que el protocolo TCP o UDP le brindan a cada protocolo de aplicación, en cada caso.

A continuación se fundamenta la elección del protocolo de transporte (TCP o UDP) y el rol de cada proceso para las 5 aplicaciones requeridas:

- **1. DHCP (Dynamic Host Configuration Protocol) → Utiliza UDP (Puertos 67 Servidor / 68 Cliente):**
  Justificación: Cuando un host cliente se inicia en la red, **carece de dirección IP asignada**, no conoce la máscara de subred ni el gateway. Por ende, es matemáticamente imposible establecer una conexión TCP (que exige direcciones IP unicast en emisor y receptor para el handshake). DHCP requiere datagramas difundidos por *Broadcast* (IP destino `255.255.255.255` y MAC destino `ff:ff:ff:ff:ff:ff`) mediante transacciones cortas e independientes (Discover, Offer, Request, ACK). UDP no añade sobrecarga de conexión ni control de flujo innecesario.
- **2. HTTP (Hypertext Transfer Protocol) → Utiliza TCP (Puerto 80):**
  Justificación: La transferencia de documentos web (HTML, JavaScript, CSS, imágenes) exige **cero tolerancia a pérdidas o corrupción de bits**; la pérdida de una sola etiqueta o carácter corrompería la página o el script. TCP brinda entrega confiable, ordenamiento estricto y control de flujo adaptado a la memoria del navegador.
- **3. HTTPS (HTTP Secure over TLS/SSL) → Utiliza TCP (Puerto 443):**
  Justificación: La capa criptográfica TLS/SSL opera como una máquina de estados secuencial basada en registros cifrados en bloques o streams continuos. Si un solo segmento se perdiera, duplicara o alterara su orden, el receptor no podría descifrar los bloques posteriores debido a la desincronización de los vectores de inicialización (IV) y números de secuencia de los MACs criptográficos. Exige fiabilidad absoluta e incondicional en Capa 4.
- **4. FTP (File Transfer Protocol) → Utiliza TCP (Puertos 21 Control / 20 o Dinámico Datos):**
  Justificación: FTP implementa una arquitectura de dos conexiones paralelas: una conexión de control (puerto 21) para autenticación y comandos de estado, y una conexión de datos (puerto 20 o dinámico negociado en PASV) para transferir archivos binarios completos. Cualquier pérdida de bytes corrompería irreversiblemente los archivos descargados o subidos, haciendo indispensable la garantía de confiabilidad y reordenamiento de TCP.
- **5. DNS (Domain Name System) → Utiliza UDP (Puerto 53) primario / TCP (Puerto 53) secundario:**
  Justificación:
  - *Uso primario de UDP:* Las consultas de nombres constan de un único mensaje de pregunta y un único mensaje de respuesta. Utilizar TCP requeriría 3 paquetes de handshake + 4 de teardown para resolver un nombre, triplicando la latencia y sobrecargando masivamente los servidores raíz y TLD de Internet.
  - *Uso de TCP:* Se utiliza cuando la respuesta excede los 512 bytes del datagrama UDP tradicional (o el buffer negociado en EDNS0) y el servidor activa el bit TC (*Truncated*), forzando al cliente a reintentar por TCP. También se emplea obligatoriamente para transferencias de zona completa (*Zone Transfers* - AXFR/IXFR) entre servidores primarios y secundarios, donde se requiere transferir bases de datos íntegras sin errores.

b) Para cada proceso o servicio, identifique los campos y parámetros (valores) del encabezado de TCP / UDP que intervienen, tanto en el Tx como en el Rx, indicando un ejemplo de cada caso:

Proceso / Servicio	Protocolo	Campos en Tx (Emisor)	Campos en Rx (Receptor)	Ejemplo con Datos Reales de la Captura
1. Segmentación	**TCP**	Opción `MSS` (Maximum Segment Size = 1460 B en SYN), Carga útil `Len`.	Opción `MSS` (1440 B en SYN-ACK), ensambla segmentos hasta agotar el flujo.	**Servidor fragmenta respuesta HTTP de 4268 B en 3 segmentos: 1440 B (Trama 7) + 1440 B (Trama 8) + 1388 B (Trama 9).**
	UDP	No segmenta en L4 (toma el bloque completo hasta 65535 B).	No segmenta (delega la fragmentación a IP en L3 si excede la MTU).	**Consulta DNS enviada en un único datagrama atómico de 45 B (UDP Length = 45 B).**
2. Ordenamiento y Reensamble	**TCP**	`Sequence Number` (32 bits, avanza por cada byte de datos enviado).	Verifica Seq recibido, almacena en buffer de reensamble y ordena correlativamente.	**Trama 7: Seq=1 (1440 B) → Trama 8: Seq=1441 (1440 B) → Trama 9: Seq=2881 (1388 B).**
	UDP	Inexistente (no tiene campo de secuencia).	Inexistente (entrega a la aplicación en el orden de llegada).	**No posee campos en la cabecera de 8 bytes de UDP.**
3. Multiplexación	**TCP**	`Source Port` (16 bits) = 65300, `Destination Port` = 80.	Invierte simétricamente: `Source Port` = 80, `Destination Port` = 65300.	**Permite asociar la respuesta al socket exacto del navegador web del cliente.**
	UDP	`Source Port` = 56459, `Destination Port` = 53.	Invierte: `Source Port` = 53, `Destination Port` = 56459.	**Permite encaminar la respuesta DNS al proceso local que lanzó la resolución de nombres.**
4. Encapsulamiento	**TCP**	`Data Offset` (4 bits, ej. 5 = 20B base o 8 = 32B con opciones), `Checksum`.	Lee `Data Offset` para conocer inicio del payload de aplicación.	**Trama 5: Cabecera TCP = 20 B, Payload HTTP = 95 B → Total Segmento = 115 B.**
	UDP	`Length` (16 bits, cabecera 8B + datos) = 45 B, `Checksum` = 0xd12b.	Comprueba `Length` y valida `Checksum` con pseudoencabezado IP.	**Trama 1 DNS: UDP Length = 45 B (8 B cabecera + 37 B consulta DNS).**
5. Conexión	**TCP**	Flags de control: `SYN`, `ACK`, `FIN`, `RST`.	Evalúa flags y transiciona máquina de estados finitos (ESTABLISHED, CLOSE_WAIT).	**Trama 1: SYN (inicio) → Trama 3: SYN-ACK → Trama 11: FIN-ACK (cierre).**

Proceso / Servicio	Protocolo	Campos en Tx (Emisor)	Campos en Rx (Receptor)	Ejemplo con Datos Reales de la Captura
	UDP	Inexistente (sin flags de conexión).	Inexistente (sin máquina de estados).	**Tráfico puramente no orientado a la conexión.**
6. Confiabilidad	**TCP**	Temporizadores de retransmisión (RTO), registro de paquetes sin confirmar.	`Acknowledgment Number` (32 bits, confirma hasta el byte n y solicita n+1).	**Trama 10: Ack = 4269 confirmando los 4268 bytes recibidos del servidor. Retransmisión SYN Trama 2.**
	UDP	Inexistente (sin acuses de recibo ni retransmisiones).	Inexistente (no confirma recepción).	**Si se pierde la consulta DNS, el cliente reintenta a nivel de aplicación tras un timeout.**
7. Control de Flujo	**TCP**	Respeta la ventana del receptor: no envía más bytes que `Window Size`.	Informa espacio libre de buffer: campo `Window Size` (16 bits) + `Window Scale`.	**Trama 1: W1 = 64240 B. Trama 3: W2 = 26883 B. Trama 4: Ventana cliente actualizada a 263424 B.**
	UDP	Inexistente (transmite a la tasa que impone la aplicación).	Inexistente (no puede frenar al emisor).	**Si el receptor no da abasto procesando datagramas, se descartan silenciosamente.**
8. Control de Errores	**TCP**	`Checksum` (16 bits) calculado sobre pseudoencabezado + cabecera + payload.	Verifica Checksum; si falla, descarta el segmento y no envía ACK (fuerza retransmisión).	**Trama 5: Checksum = 0x937c [validado con éxito por el receptor].**
	UDP	`Checksum` (16 bits) calculado sobre pseudoencabezado IP + cabecera UDP + datos.	Verifica Checksum; si falla, descarta silenciosamente (sin aviso al emisor).	**Trama 1: Checksum = 0xd12b. Trama 2: Checksum = 0xcba4 [validado].**

c) Cálculo detallado para determinar el encapsulamiento de un segmento UDP y otro TCP:

Se realiza el cálculo matemático riguroso de encapsulamiento capa por capa comparando la trama TCP (petición HTTP GET) contra la trama UDP (consulta DNS):

1. Encapsulamiento del Segmento TCP (Petición HTTP GET - Trama 5):

- **Capa 7 - Aplicación (HTTP GET):** `GET / HTTP/1.1 Host: neverssl.com ...` = **95 bytes** de carga útil.

- **Capa 4 - Transporte (TCP):** Cabecera base sin opciones: Puerto Origen (2 B) + Puerto Destino (2 B) + Sequence Number (4 B) + Acknowledgment Number (4 B) + Data Offset/Reserved/Flags (2 B) + Window Size (2 B) + Checksum (2 B) + Urgent Pointer (2 B) = **20 bytes**.
- **Capa 3 - Red (IPv4):** Cabecera IPv4 estándar: Versión/IHL (1 B) + DSCP/ECN (1 B) + Total Length = 135 B (2 B) + Identification (2 B) + Flags/Fragment Offset (2 B) + TTL (1 B) + Protocol = 6 TCP (1 B) + Header Checksum (2 B) + IP Origen (4 B) + IP Destino (4 B) = **20 bytes**.
- **Capa 2 - Enlace de Datos (Ethernet II):** MAC Destino (6 B) + MAC Origen (6 B) + EtherType 0x0800 (2 B) = **14 bytes** (más 4 bytes de cola FCS verificada por el hardware = 18 B).

Cálculo de Longitud Total de la Trama TCP:

```
Longitud Trama (Wire) = 14 (Eth) + 20 (IP) + 20 (TCP) + 95 (HTTP) = 149 Bytes (153 B con FCS)
Eficiencia de Datos Útiles (Payload Ratio) = 95 / 149 = 63.76 %
Sobrecarga Total de Protocolos (Overhead)  = 54 / 149 = 36.24 %
(los 20 B de la cabecera TCP representan el 13.42 % de la trama total)
```

2. Encapsulamiento del Segmento UDP (Consulta DNS Query - Trama 1):

- **Capa 7 - Aplicación (DNS Query):** Encabezado DNS (Transaction ID, Flags, Questions Count) + Consulta por registro A de `www.frba.utn.edu.ar` = **37 bytes** de carga útil.
- **Capa 4 - Transporte (UDP):** Encabezado mínimo fijo: Source Port (2 B) + Destination Port (2 B) + Length (2 B) + Checksum (2 B) = **8 bytes**.
- **Capa 3 - Red (IPv4):** Cabecera IPv4 estándar (Total Length = 65 bytes, Protocol = 17 UDP) = **20 bytes**.
- **Capa 2 - Enlace de Datos (Ethernet II):** Cabecera MAC Ethernet II = **14 bytes** (más 4 bytes de FCS = 18 B).

Cálculo de Longitud Total de la Trama UDP:

```
Longitud Trama (Wire) = 14 (Eth) + 20 (IP) + 8 (UDP) + 37 (DNS) = 79 Bytes (83 B con FCS)
Eficiencia de Datos Útiles (Payload Ratio) = 37 / 79 = 46.84 %
Sobrecarga Total de Protocolos (Overhead)  = 42 / 79 = 53.16 %
(los 8 B de la cabecera UDP representan solo el 10.13 % de la trama total)
```

Conclusión Comparativa del Encapsulamiento:
El encabezado UDP introduce una sobrecarga mínima constante de **8 bytes**, lo que representa una **reducción del 60% en sobrecarga de transporte** frente a los 20 bytes mínimos de TCP (o hasta 60 bytes cuando TCP incluye opciones de sincronización como MSS, Window Scale, SACK y Timestamps). Esto optimiza la velocidad y latencia en transacciones cortas (DNS, DHCP, streaming de audio/video en tiempo real), mientras que TCP justifica sus 20 bytes adicionales aportando control de congestión, control de flujo, ordenamiento y entrega confiable garantizada.

CRITERIO DE AUTOEVALUACION

Verificación de Objetivos Alcanzados:

1. **Ejecución experimental completa:** Se capturaron y resguardaron satisfactoriamente en la carpeta `wireshark/TL6/capturas/` las trazas reales de los 5 protocolos analizados:
   - `3a_tcp_http_session.pcapng`: Sesión TCP HTTP completa (handshake de 3 vías, transferencia segmentada y liberación en 4 vías).
   - `3b_udp_dns_query.pcapng`: Consulta y respuesta DNS atómica sobre datagramas UDP.
   - `dhcp.pcap`: Proceso DORA sobre UDP puertos 67 y 68.
   - `3c_tcp_https_session.pcapng`: Sesión HTTPS/TLS sobre TCP puerto 443.
   - `3d_tcp_ftp_session.pcapng`: Sesión FTP con canal de control (21) y canal de datos (1040).
2. **Respaldo de respuestas teóricas y experimentales:** Cada pregunta del cuestionario fue respondida exhaustivamente con texto explicativo, fundamentación teórica de acuerdo a los estándares RFC y valores numéricos exactos extraídos de las capturas de laboratorio.